Predict, Then Choose: A Latent World Model for Language-Guided Robot Pushing

Team: Wasiq Malik · [Teammate name]

Code: github.com/humanwhoexplores/RoboJEPA, branch `wm/midterm-jepa-vs-direct`

ABSTRACT A robot told to "push the cube onto the green square" must choose, among many possible motions, one that makes the instruction true. We test whether a JEPA-style latent world model, which predicts the visual features after each push and checks the prediction against the instruction, chooses better than a classifier that scores each plan directly from the same inputs. A simulated Franka Panda (robosuite) ranks 48 one- and two-push plans per scene; both methods share frozen DINOv2 and MiniLM encoders. We have built the task, data pipeline, both models and a 3-seed evaluation. With 400 training scenes, JEPA selects a successful plan 0.74 ± 0.02 of the time against 0.55 ± 0.05 for the direct classifier, and 0.67 vs 0.42 on two-push plans. On cube positions outside the training range JEPA falls behind (0.38 vs 0.47). Next we test whether a pretrained world model (Meta's RoboJEPA) and more diverse data close that gap.

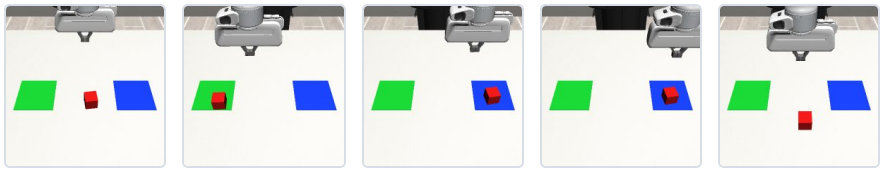
2 · Problem Statement and Goal Recap

Vision-language-action models map an image and an instruction straight to motor commands. World-model approaches such as JEPA [1, 2] and DINO-WM [3] instead learn to predict the consequences of an action in a learned feature space and plan by searching over predicted outcomes. Our question is narrow and controlled: **given the same inputs, encoders and parameter budget, does explicitly predicting the next state help a robot pick the action that satisfies a language instruction?** We study it on a tabletop pushing task where the instruction names a target region either by side ("left", "right") or by color ("blue", "green"). The colors swap between scenes, so color instructions can only be solved by reading the image.

Objectives committed to for the first half of the term, and their status:

Objective	Status
O1. Simulated language-guided push task with held-out paraphrases and an out-of-distribution (OOD) test split	DONE 400/50/100/60 scenes
O2. JEPA (predictor + goal scorer) and direct-classifier baseline on shared frozen encoders	DONE trained from scratch
O3. Comparison with a data-size sweep (50–400 scenes) and ≥ 3 seeds	DONE 3 seeds at 100 and 400
O4. Hypotheses: JEPA wins in-distribution, gains most on multi-step plans, generalizes better OOD, needs less data	PARTIAL first two supported, last two not

3 · Methods and Implementation to Date



START
Left region is green, right is blue.

1 PUSH → LEFT
Succeeds for "left" and "green".

2-PUSH PLAN, AFTER PUSH 1
Cube already reaches the right region.

AFTER PUSH 2 → RIGHT
Succeeds for "right" and "blue".

1 PUSH → NO REGION
Fails for every instruction.

Figure 1. One test scene from the 224 px policy camera. A push is a start and an end point on the table; a plan has one or two pushes. The arm retracts after each push, so every frame is a clean decision point.

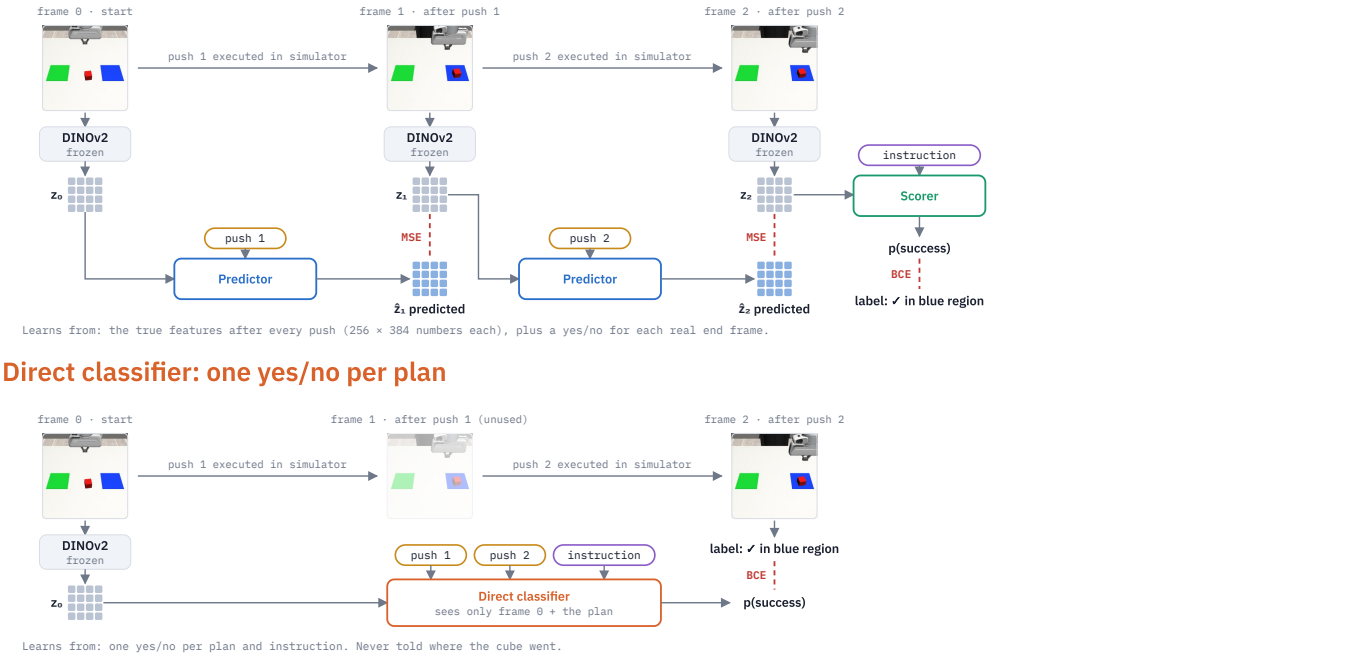


Figure 2. Training signals. JEPA's predictor gets a dense target (256 × 384 features) after every push and its scorer gets a yes/no per end frame; the direct classifier gets one yes/no per plan. At test time each method scores all 48 candidate plans from the current frame and the robot executes the highest-scoring one, open loop.

Key design decisions.

- **Task.** MuJoCo + robosuite, Franka Panda with an operational-space controller (20 Hz). Success means the cube rests inside the named 13 × 13 cm region after the plan.
- **Candidates are goal-blind.** Plans are generated around a noisy cube estimate (σ = 1 cm, standing in for a detector): 8 directions × 3 lengths as single pushes, plus 24 two-push plans. The generator never sees the instruction, so ranking is the whole problem.
- **Shared frozen encoders.** DINOv2-small [4] gives 256 patch features of size 384 per image; all-MiniLM-L6-v2 [5] embeds the instruction.
- **JEPA.** Predictor: 6-layer transformer over a push token and 256 patch tokens, outputs the change in features (zero-initialized residual head), MSE loss, rolled out push by push. Scorer: 2-layer transformer over instruction + patches, weighted BCE on true end frames. Plan score = scorer(predicted end features).
- **Direct classifier.** 6-layer transformer over instruction + 2 push tokens + start patches, weighted BCE. Same width, same data, no future prediction.
- **Oracle.** The JEPA scorer applied to the *true* end features. It is an upper bound that separates prediction error from scoring error.
- **Training and evaluation.** Every outcome is labeled against all 4 targets. Fixed step budgets at every data size (predictor and direct 15k, scorer 5k), batch 64, AdamW 3e-4, bf16; checkpoint by validation loss or AUC. Metric: **selection success**, the share of solvable scene–instruction pairs where the top-ranked plan succeeds (random ≈ 0.10). Ablations: data size, plan length, held-out phrasings, baseline tuning.

Reproducibility. One AMD RX 9070 XT (16 GB, ROCm 7.2), PyTorch 2.13 (ROCm build), MuJoCo 3.3.0, robosuite 1.5.2, HuggingFace Transformers. Collecting all 610 scenes takes ~30 min with 5 workers; a 15k-step predictor run takes ~7 min. `python -m robojepa.collect` then `scripts/run_midterm.sh` reproduces every number below.

4 · Preliminary Experiments and Results

Setup. Training: 400 scenes with 16 random plans each (6,400 plans, ~18% successful); validation: 50 scenes. Test: 100 in-distribution scenes and 60 OOD scenes in which the cube starts in a 4 cm strip nearer the camera that no training scene covers; each test scene is paired with 8 instructions (5 training phrasings and 3 held-out paraphrases) and ranked over the same 48 plans.

Training scenes · test split	Plans	Random	Direct	JEPA (ours)	Oracle
400 scenes · in-distribution					
all plans	48	0.10	0.55 ±0.05	**0.74 ±0.02**	0.97 ±0.02
single-push plans only	24	0.10	0.86 ±0.01	0.86 ±0.03	0.98 ±0.02
two-push plans only	24	0.13	0.42 ±0.03	**0.67 ±0.02**	0.97 ±0.02
held-out paraphrases	48	0.10	0.51 ±0.07	**0.68 ±0.05**	0.91 ±0.06
400 scenes · OOD cube positions					
all plans	48	0.09	**0.47 ±0.01**	0.38 ±0.04	0.93 ±0.01
100 scenes					
in-distribution, all plans	48	0.10	0.49 ±0.06	**0.60 ±0.02**	0.95 ±0.01
OOD, all plans	48	0.09	0.40 ±0.08	0.38 ±0.08	0.89 ±0.01

Table 1. Selection success, mean ± std over 3 seeds. JEPA beats the direct classifier in-distribution by 19 points at 400 scenes and 11 at 100, well outside seed noise. The whole gain comes from two-push plans: on single pushes the two tie. Off-distribution, JEPA is behind at 400 scenes and tied at 100.

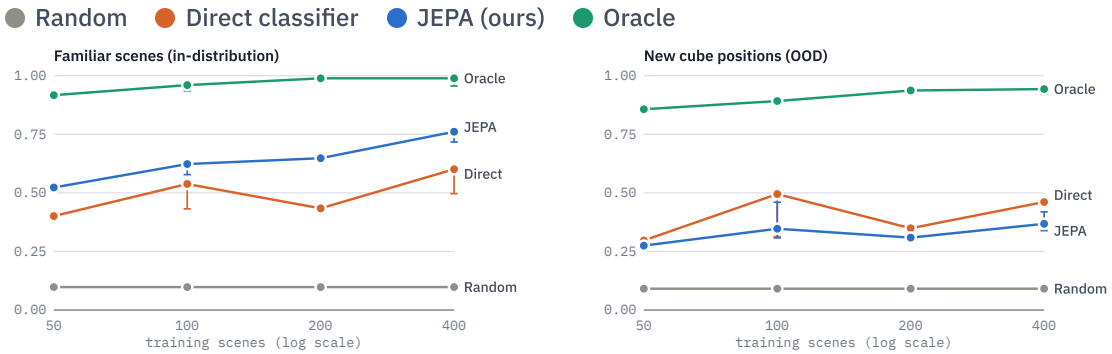


Figure 3. Selection success vs number of training scenes (seed 0; error bars at 100 and 400 are ± std over 3 seeds). JEPA leads in-distribution at every size, but the gap does not widen as data shrinks, so we cannot yet claim better data efficiency.

Interpretation.

- **What worked:** predicting first helps when the plan is compositional. A single push only requires learning which direction leads to which region. For a two-push plan, the second push only makes sense given where the first leaves the cube; the predictor rolls forward one push at a time, while the direct classifier has to learn that composition implicitly.
- **Language is not the bottleneck.** The oracle picks a working plan 89–99% of the time in every setting, and JEPA keeps its lead on paraphrases it never saw (0.68 vs 0.51). The gap between JEPA and the oracle is world-model error.
- **What didn't work:** OOD generalization. When the cube starts a few centimetres outside the training range, the predictor's feature error rises by ~80% (Table 2), its ranking AUC drops from 0.95 to 0.84, and the direct classifier wins.
- **Surprise:** 4× more data (100 → 400 scenes) improved JEPA in-distribution by 14 points but did not move its OOD score (0.38 → 0.38). More of the same data does not fix extrapolation.

Check (400 scenes)	Result	Why it matters
Floor and ceiling	Random 0.10; oracle 0.97 ID / 0.93 OOD	Task is solvable from features; the scorer reads language correctly
Predictor vs “nothing moves” baseline (feature MSE)	ID 0.30 vs 1.32 (1 push), 0.47 vs 1.76 (2 pushes); OOD 0.54 vs 1.92	The predictor learns real dynamics (−75% error) but degrades off-distribution
Direct baseline tuning (seed 0)	default 0.60; lr 1e-4 0.62; batch 32 0.51; JEPA 0.76	The JEPA lead is not an under-tuned baseline
Ranking AUC over all candidates	ID: JEPA 0.95, direct 0.94; OOD: 0.84 vs 0.93	Matches the selection results without depending on the top-1 pick
Instruction use at 50 scenes (first pipeline)	Every instruction scored the same; oracle only 0.55	Caught a label bug; fixed (Section 5)
Cost of scoring 48 plans	JEPA 14.6 ms vs direct 6.3 ms; 7.0M vs 5.1M trained params	JEPA is 2.3× slower, still far within real-time

Table 2. Validity checks and quick ablations. The JEPA advantage survives baseline tuning and is consistent across metrics; the OOD loss is explained by world-model error rather than by the language scorer.

5 · Roadblocks, Risks, and Mitigations

Roadblock	Root cause and action taken	Status
Models ignored the instruction at low data	Each outcome was labeled for one instruction only, and epoch-based training gave small datasets few updates. We now label every outcome against all 4 targets and train a fixed number of steps at every data size; the 50-scene oracle went from 0.55 to 0.92.	GREEN fixed
JEPA loses on unseen cube positions	The 5M predictor trained on 400 scenes interpolates but does not extrapolate. Tried: 4× more data (no OOD gain). Next: a pretrained world model and more diverse scenes (Section 6).	RED open, main risk
Toolchain on a new GPU	robosuite 1.5.2 breaks with MuJoCo 3.17 (pinned 3.3.0); pytorch.org ROCm wheels are 6 GB, so we used vLLM’s ROCm wheels that link system ROCm; headless rendering via EGL.	GREEN resolved
Disk space	The disk was full (cached data and features 4 GB, teammate dataset 3.2 GB). Cleared 25 GB of unused model caches; features are stored in fp16.	YELLOW watch
Integrating the teammate’s richer dataset	It uses a kinematic paddle instead of the Panda, a different action format, and goal-aware candidates where a random “ideal” push already succeeds 89% of the time. Plan: use it to pretrain the predictor, keep our goal-blind Panda tests.	YELLOW in progress

Risk	Prob. / impact	Contingency
Pretrained RoboJEPA does not fit our 16 GB GPU or action format	Medium / High	Use the 22M predictor locally, larger ones on Colab; re-express pushes as end-effector trajectories logged in simulation
Pretraining does not close the OOD gap	Medium / Medium	Report it as a negative result; add OOD scenes to training as an upper-bound reference
Offline ranking overstates real success	Low / Medium	Closed-loop execute-and-replan evaluation on a subset of test scenes
Name clash with Meta’s RoboJEPA confuses readers	High / Low	Rename our project before the final report

6 · Updated Plan and Milestones

Delta from the original timeline. The core comparison finished on schedule. Two things changed scope: Meta released RoboJEPA in October 2026 [6], a family of action-conditioned predictors (22M–8B) on a frozen V-JEPA 2.1 encoder trained on Panda data, which gives us a pretrained world model to test instead of only scaling our own; and a teammate generated a larger, more varied scene dataset (1,000 scenes, 78 goal types). Closed-loop evaluation moves to a stretch goal.

Target date	Milestone	Done when
Nov 9	Run RoboJEPA-22M on our scenes; scorer retrained on V-JEPA features	V-JEPA oracle ≥ 0.90 on in-distribution test
Nov 16	Zero-shot RoboJEPA + our scorer vs our JEPA vs direct	All three ranked on the same 48 candidates, ID and OOD
Nov 23	Pretrain our predictor on the teammate dataset, fine-tune on Panda data	3 seeds at 100 and 400 scenes
Nov 30	Fine-tune RoboJEPA-22M at 50/100/400 scenes; larger predictors on Colab	Data-efficiency curve with latency and parameter counts
Dec 7	Closed-loop replanning (stretch); final report and rename	Executed success on ≥ 50 test scenes

Evaluation plan for the final. Same goal-blind candidates and selection-success metric, plus ranking AUC, latency and parameter count; stronger baselines (pretrained world model zero-shot and fine-tuned, direct classifier on the same pretrained features); 3 seeds throughout.

Success criteria. (1) A world-model variant reaches OOD selection success ≥ 0.47, matching the direct classifier, while keeping in-distribution success ≥ 0.74. (2) Fine-tuned at 100 scenes, it matches what our from-scratch JEPA needs 400 scenes for (0.74). (3) Scoring 48 plans stays under 100 ms on our GPU for the 22M predictor.

7 · References

1. M. Assran et al. Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA). CVPR 2023.
2. M. Assran et al. V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning. arXiv:2506.09985, 2025.
3. G. Zhou et al. DINO-WM: World Models on Pre-trained Visual Features Enable Zero-shot Planning. arXiv:2411.04983, 2024.
4. M. Oquab et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR 2024. Model: facebook/dinov2-small.
5. N. Reimers, I. Gurevych. Sentence-BERT. EMNLP 2019. Model: sentence-transformers/all-MiniLM-L6-v2.
6. Meta FAIR. RoboJEPA: action-conditioned latent world models for robot planning. arXiv:2610.10515, 2026.
7. Y. Zhu et al. robosuite: A Modular Simulation Framework and Benchmark for Robot Learning. arXiv:2009.12293, 2020.
8. E. Todorov, T. Erez, Y. Tassa. MuJoCo: A Physics Engine for Model-Based Control. IROS 2012.
9. A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.